



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Tarea 10

ALUMNO

**Pérez Takayanagui Louis Emilio
322311419**

MATERIA

Estructura y Programación de Computadoras

23 de abril de 2026

1. Implementar los ejercicios de programación “Código de ejemplo 2”, “Código de ejemplo 3”.

- Implementación del Código de ejemplo 2 y su respectiva validación en el Turbo Debugger:

```

1 title demostracion de and or y xor
2 model small
3 stack 16
4
5 dataseg
6     exCode db 0
7     sourceWord dw 0abh
8     wordmask dw 0cfh
9
10 codeseg
11 Start:
12     mov ax,@data
13     mov ds,ax
14     mov ax,[sourceWord]
15     mov bx,ax
16     mov cx,ax
17     mov dx,ax
18     and ax,[wordMask]
19     or bx,[wordMask]
20     xor cx,[wordMask]
21     xor dx,dx
22 Exit:
23     mov ah,04ch
24     mov al,[exCode]
25     int 21h
26     end Start

```

Listing 1: Código de ejemplo 2.

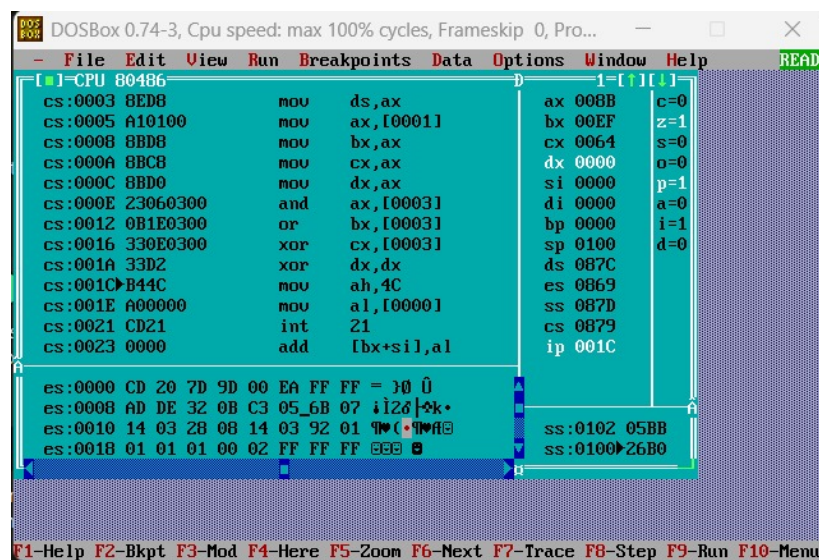


Figura 1: Captura del Turbo Debugger del código de ejemplo 2.

- Implementación del **Código de ejemplo 3** y su respectiva validación en el Turbo Debugger:

```

1 title Demostracion de add, sub, inc, dec
2 model small
3 stack 16
4
5 dataseg
6     exCode db 0
7     count dw 1
8
9 codeseg
10 Start:
11     mov ax,@data
12     mov ds,ax
13     mov ax,4
14     mov bx,2
15     add ax,bx
16     mov cx,8
17     add cx,[count]
18     add [count],cx
19     inc [count]
20     dec [count]
21     inc ax
22     dec cx
23 Exit:
24     mov ah,4ch
25     mov al,[exCode]
26     int 21h
27     end Start

```

Listing 2: Código de ejemplo 3.

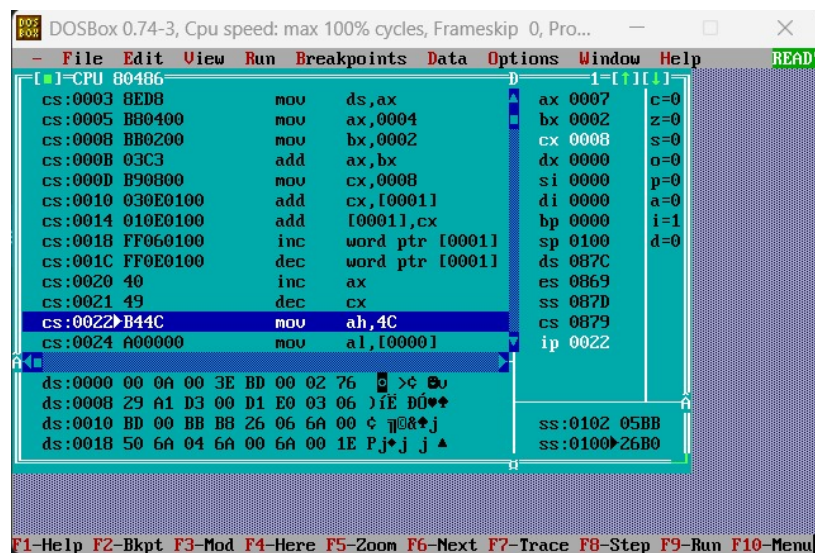


Figura 2: Captura del Turbo Debugger del código de ejemplo 3.

2. Implementar el código que realiza la copia de una cadena a otra.

- Implementación del Código que realiza la copia de una cadena a otra y su respectiva validación en el Turbo Debugger:

```

1 title Copia cadena
2 model small
3 stack 16
4 dataseg
5 original db "hola_mundo"
6 destino db "aaaaaaaa"
7 exCode db 0
8
9 codeseg
10 Start:
11     mov ax, @data
12     mov ds, ax
13     mov cx, 10
14     mov si, 0
15
16 Iteracion:
17     mov al, [original + si]
18     mov [destino + si], al
19     inc si
20     loop Iteracion
21
22 Exit:
23     mov ah, 4ch
24     mov al, [exCode]
25     int 21h
26 end Start

```

Listing 3: Código de Copiar Cadena.

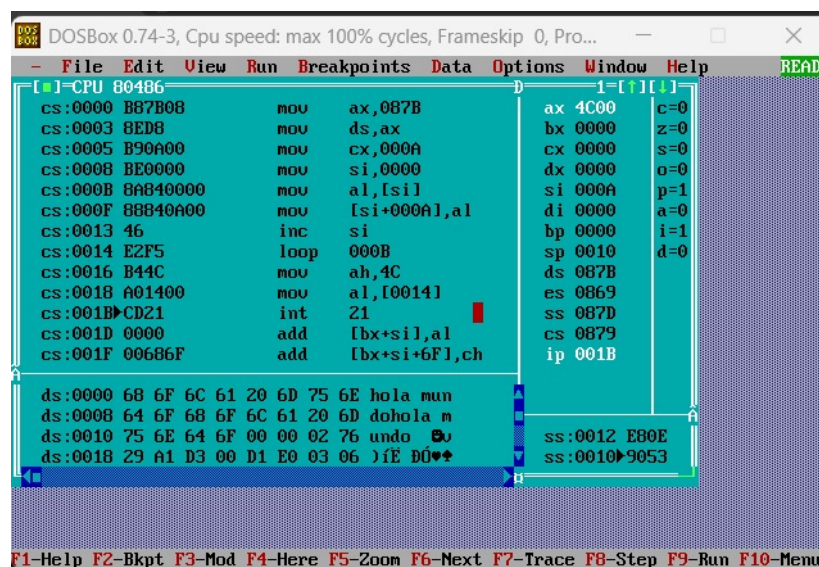


Figura 3: Captura del Turbo Debugger del código que copia una cadena a otra.

3. “SoYfElIz” a “sOyFeLiZ”, en versión DEBUG y TASM.

- Implementacion del Código que invierte mayúsculas a minúsculas y viceversa y su respectiva validación en el Turbo Debugger:

```

1  title Invertir minusculas a mayusculas
2  model small
3  stack 16
4
5  dataseg
6      cadena db "SoYfElIz"
7      tam dw 8
8      exCode db 0
9
10 codeseg
11 Start:
12     mov ax, @data
13     mov ds, ax
14     mov cx, [tam]
15     mov si, 0
16
17 Invertir:
18     mov al, [cadena + si]
19     xor al, 20h
20     mov [cadena + si], al
21     inc si
22     loop Invertir
23
24 Exit:
25     mov ah, 4ch
26     mov al, [exCode]
27     int 21h
28 end Start

```

Listing 4: Código de Invertir Mayúsculas y Minúsculas en TASM.

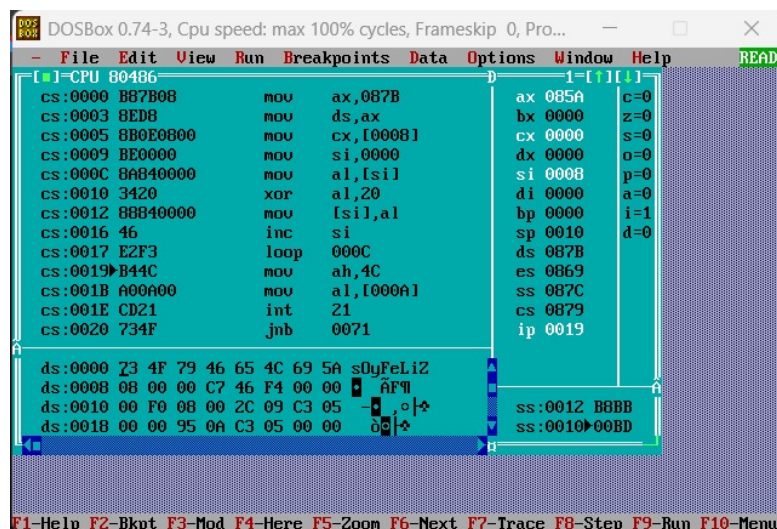


Figura 4: Captura del Turbo Debugger del código que invierte mayúsculas y minúsculas.

4. Suma de arreglos en TASM:

- Implementación del Código que hace la suma de dos arreglos y su respectiva validación en el Turbo Debugger:

```
1 title "Suma de arreglos"
2 model small
3 stack 16
4
5 dataseg
6     A db 1, 2, 3, 4, 5, 6
7     B db 0ah, 0bh, 0ch, 0dh, 0eh, 0fh
8     C db 0, 0, 0, 0, 0, 0
9     exCode db 0
10
11 codeseg
12 Start:
13     mov ax, @data
14     mov ds, ax
15
16     mov cx, 6
17     mov si, 0
18
19 Suma:
20     mov al, [A + si]
21     add al, [B + si]
22     mov [C + si], al
23     inc si
24     loop Suma
25
26 Exit:
27     mov ah, 4ch
28     mov al, [exCode]
29     int 21h
30
31 end Start
```

Listing 5: Código de la suma de dos arreglos.

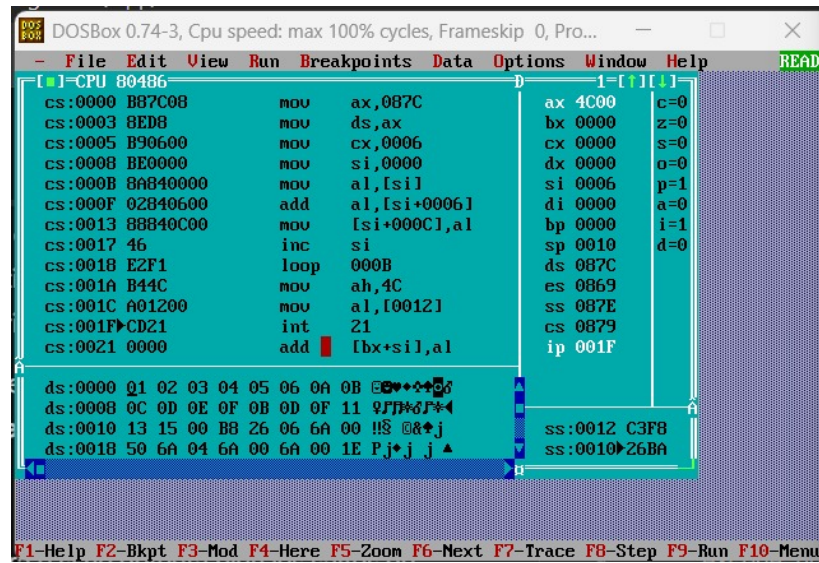


Figura 5: Captura del Turbo Debugger del código que la suma de dos arreglos.